SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Question 1: Water Jug Problem

1. Problem Understanding

- Two jugs: a **4-gallon** jug and a **3-gallon** jug, with no measuring marks. A pump supplies unlimited water.
- Allowed operations: fill a jug from the pump, empty a jug onto the ground, pour from one jug into the other.
- **Goal:** exactly **2 gallons in the 4-gallon jug**.

2. Problem Formulation

- **State:** ordered pair (x, y), where x = water in the 4-gallon jug ($0 \le x \le 4$) and y = water in the 3-gallon jug ($0 \le y \le 3$).
- **Initial state:** (0, 0).
- **Goal state:** (2, n) for any n; here we reach (2, 0).
- **Path cost:** each operation costs 1 (we want the fewest steps).

3. Method Selection

State-space search using production rules. **Breadth-First Search (BFS)** is chosen because every action has equal cost, so BFS is complete and returns the shortest sequence of operations.

4. Production Rules (Operators)

No.	Condition	Action	Result
1	$x < 4$	Fill the 4-gallon jug from the pump	(4, y)
2	$y < 3$	Fill the 3-gallon jug from the pump	(x, 3)
3	$x > 0$	Empty the 4-gallon jug on the ground	(0, y)
4	$y > 0$	Empty the 3-gallon jug on the ground	(x, 0)
5	$x + y \ge 4$ and $y > 0$	Pour from 3-gal into 4-gal until the 4-gal is full	$(4, y - (4 - x))$
6	$x + y \ge 3$ and $x > 0$	Pour from 4-gal into 3-gal until the 3-gal is full	$(x - (3 - y), 3)$
7	$x + y \le 4$ and $y > 0$	Pour all water from 3-gal into 4-gal	$(x + y, 0)$
8	$x + y \le 3$ and $x > 0$	Pour all water from 4-gal into 3-gal	$(0, x + y)$

5. Solution Process (Step by Step)

Step	Rule applied	Explanation	State (x, y)
0	–	Both jugs empty (initial state)	(0, 0)
1	Rule 2	Fill the 3-gallon jug from the pump	(0, 3)
2	Rule 7	Pour all 3 gallons into the 4-gallon jug	(3, 0)
3	Rule 2	Fill the 3-gallon jug again	(3, 3)

Step	Rule applied	Explanation	State (x, y)
4	Rule 5	Pour from 3-gal into 4-gal until full; the 4-gal needs only 1 gallon, leaving 2 in the 3-gal jug	(4, 2)
5	Rule 3	Empty the 4-gallon jug on the ground	(0, 2)
6	Rule 7	Pour the 2 gallons from the 3-gal jug into the 4-gal jug	**(2, 0)** – GOAL

6. Alternative Solution (also 6 steps)

(0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → **(2,3)**. Here the 4-gallon jug again holds exactly 2 gallons.

7. Interpretation of Result

The 4-gallon jug contains exactly **2 gallons** after **6 operations**. Verification: water is only added by the pump and lost on the ground; the total at step 4 is 4 + 2 = 6 gallons, and after discarding 4 gallons the remaining 2 gallons are moved into the 4-gallon jug. Since BFS explores states level by level, no solution shorter than 6 steps exists.

Question 2: Mars Rover – PEAS Analysis

A Mars rover is an autonomous robotic agent that moves over the Martian surface, collects soil/rock samples, analyses them with onboard instruments and sends the results to Earth. It is described using the **PEAS** framework (Performance, Environment, Actuators, Sensors).

i. Percepts of the agent

Percepts are what the rover receives through its sensors:

- **Camera images** (stereo/panoramic) of terrain, rocks, and obstacles.
- **Spectrometer / chemical-analysis readings** of soil and rock samples (composition, minerals, organic traces).
- **Terrain and obstacle data** from lidar/proximity and slope (tilt) sensors.
- **Position and motion data:** GPS-like localisation, gyroscope/IMU orientation, wheel odometer.
- **Weather data:** temperature, wind speed, atmospheric pressure, radiation, dust level.
- **Internal status:** battery/power level, solar-panel output, instrument and wheel health, drill force feedback.
- **Commands/messages** received from the Earth control centre.

ii. Characteristics of the operating environment

Property	Type	Reason
Observability	Partially observable	Sensors see only the nearby area; the far terrain and sample contents are hidden.
Agents	Single agent	The rover acts alone (Earth operators only send high-level goals).
Determinism	Stochastic	Wheel slip, loose sand, and dust storms make outcomes uncertain.
Episodic/Sequential	Sequential	Current moves and energy use affect later decisions.
Static/Dynamic	Dynamic	Weather, dust, light, and temperature change while the rover acts or deliberates.
Discrete/Continuous	Continuous	Position, speed, temperature, and time are continuous values.
Known/Unknown	Largely unknown	The terrain is unmapped; the rover must learn as it explores.

Additional features: a harsh environment (extreme cold, radiation) and a **communication delay of about 4–24 minutes** with Earth, so the rover must act autonomously.

iii. Actions the agent can take

- Move forward/backward, turn left/right, stop, climb or reroute around obstacles.
- Take photographs and scan the terrain.
- Deploy the robotic arm, drill, scoop/collect and store a sample.
- Analyse samples with spectrometers and other instruments.

- Transmit data to Earth/orbiter and receive commands.
- Adjust solar panels, recharge, heat instruments, enter sleep/safe mode.

iv. Performance measure

- Number and **scientific value** of samples correctly collected and analysed.
- Area of Mars explored and number of targets of interest reached.
- Accuracy and completeness of the data transmitted to Earth.
- **Safety:** no collisions, falls, stuck wheels, or instrument damage.
- **Energy and time efficiency** and the length of the mission (rover lifetime).

PEAS summary

Performance	Environment	Actuators	Sensors
Samples analysed, area explored, data quality, safety, energy efficiency	Martian surface: rocks, sand, slopes, dust storms, temperature extremes	Wheels, steering, robotic arm, drill, scoop, antenna, solar panel	Cameras, spectrometers, lidar, IMU, temperature/wind sensors, battery monitor

v. Most suitable agent architecture and why

A **goal-based agent combined with a utility-based (learning) element**, organised as a hybrid/layered architecture, is most suitable.

- **Model-based reflex layer (bottom):** handles immediate safety, such as stopping at a cliff edge or avoiding a rock. The rover cannot see the whole environment (partial observability), so it keeps an internal model of the map and its own state.
- **Goal-based layer:** the rover has explicit goals (reach a site, collect a sample), and it must plan sequences of actions to achieve them; a simple reflex agent cannot plan.
- **Utility-based layer:** when goals conflict (a valuable sample versus battery or risk), utility lets the rover trade off science value, energy, and safety.
- **Learning element:** the terrain is unknown, so the rover improves its navigation and target selection from experience.

A simple reflex agent fails in a partially observable, sequential environment, and the long communication delay with Earth rules out purely remote control. Hence the hybrid goal/utility-based learning agent is the best choice.

Question 3: 8-Queens Problem – Search Space and Problem Formulation

1. Problem Understanding

Place 8 queens on an 8 × 8 chessboard so that **no two queens attack each other**, i.e., no two queens share the same row, the same column, or the same diagonal. The configuration in the figure is not a solution because the queen in the first column and the queen in the last column lie on the same diagonal.

2. Problem Formulation (Components)

Two formulations are possible. The **incremental formulation** is better.

Component	Naïve incremental formulation	Improved incremental formulation (recommended)
States	Any arrangement of 0 to 8 queens on the board	Arrangement of n queens (0 ≤ n ≤ 8), one per column in the leftmost n columns, with no queen attacking another
Initial state	Empty board (no queens)	Empty board (no queens)
Actions	Add a queen to any empty square	Add a queen to a square in the leftmost empty column that is not attacked by any queen already placed
Transition model	Returns the board with the queen added	Returns the board with the queen added to the chosen square
Goal test	8 queens on the board, none attacked	8 queens on the board, none attacked
Path cost	Not relevant (only the final configuration matters)	Not relevant (only the final configuration matters)
State-space size	$64 \times 63 \times \ldots \times 57 \approx 1.8 \times 10^{14}$	Only **2,057** states

Complete-state formulation (alternative): start with all 8 queens on the board (one per column, $8^8 \approx 16.7$ million states) and move a queen within its column to reduce conflicts (used by local search such as hill climbing).

3. Exploring the Search Space with Search Strategies

- **Breadth-First Search (BFS):** complete, but it must store all nodes at every level, which uses huge memory for 8-Queens.
- **Depth-First Search (DFS) with backtracking:** goes deep, placing one queen per column; if no safe square exists in a column, it backtracks and moves the previous queen. Memory is only O(bm); the solution lies at depth exactly 8, so all solutions are at the same depth.
- **Depth-Limited Search (limit = 8):** a depth limit of 8 is natural since a solution needs exactly 8 placements, so it avoids infinite paths.

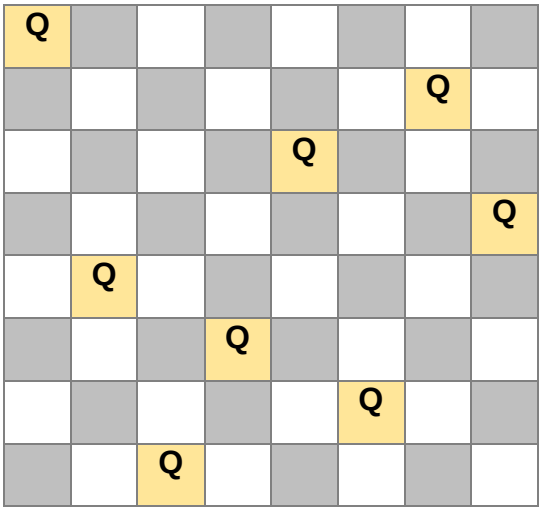
Choice: DFS/backtracking with the improved formulation is most suitable (small memory, solutions at a fixed depth, and early pruning of attacked squares).

4. Solution Process (DFS Backtracking Trace)

Queens are placed column by column, trying rows 1 to 8 in order. (Row, column) notation, with columns 1–8:

- Column 1 → row 1: Q at (1,1).
- Column 2 → rows 1 and 2 are attacked (same row / diagonal), so row 3 is chosen: (3,2).
- Column 3 → rows 1–4 are attacked, so row 5: (5,3). Column 4 → row 2: (2,4). Column 5 → row 4: (4,5).
- Column 6 → rows 6, 7 and 8 are all attacked by earlier queens (diagonals), so there is **no safe square: dead end → backtrack** to column 5 and try the next row, and so on.
- After exhaustive backtracking (column 1 stays at row 1 and column 2 eventually moves to row 5), the search places columns 3 to 8 at rows **8, 6, 3, 7, 2, 4**.

The first solution found by DFS is **(rows per column 1 to 8) = (1, 5, 8, 6, 3, 7, 2, 4)**:



5. Verification of the Solution

Column c	1	2	3	4	5	6	7	8
Row r	1	5	8	6	3	7	2	4
r − c	0	3	5	2	−2	1	−5	−4
r + c	2	7	11	10	8	13	9	12

All row numbers are distinct (no row attack), each queen is in its own column, all r − c values are distinct (no "\" diagonal attack), and all r + c values are distinct (no "/" diagonal attack). Therefore no queen attacks another.

6. Interpretation

The 8-Queens problem has **92 solutions** (12 distinct up to rotation and reflection). The improved formulation cuts the search space from about 1.8×10^{14} to 2,057 states, which is why a good formulation matters more than raw computing power.

Question 4: OLA Cab Booking – Problem-Solving Agent

1. Problem Understanding

A customer wants to travel from a source location to a destination using the OLA app and may choose a cab type (Mini, Micro, Sedan, Shared, Prime, etc.) based on comfort. The system must pick a suitable cab and a route to reach the destination.

2. Type of Agent

A **goal-based problem-solving agent** is suitable: the agent has a clear goal ("customer at destination"), formulates it as a problem, searches for a sequence of actions, and executes it. Because the choice of cab type depends on comfort, fare, and waiting time, a **utility-based element** is added to rank the cab options. The environment is observable, deterministic (approximately), and known, so the standard problem-solving (search) agent applies.

3. Problem Formulation

Component	Description
Initial state	Customer at pickup location, no cab booked
Goal state	Customer at the destination
States	(location, cab type, trip status): searching / booked / on-trip / completed
Actions	Select cab type, book cab, wait for driver, travel along a road segment, complete the trip, make the payment
Transition model	Action applied in a state gives the new location / status
Path cost	Fare + travel time (weighted sum)

4. PEAS Description

- **Performance:** low waiting time, low fare, comfort, safety, shortest route.
- **Environment:** road network, traffic, available cabs, customer preferences.
- **Actuators:** app display, booking request, route display, notification, payment.
- **Sensors:** GPS location, cab availability, traffic data, customer input.

5. Pseudocode – General Problem-Solving Agent

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty), state, goal,
  problem
  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

6. Pseudocode – OLA Cab Booking Agent

```
function OLA-CAB-AGENT(pickup, destination, preference)
  goal ← AT(destination)
  cabs ← GET-AVAILABLE-CABS(pickup)      // Mini, Micro, Sedan, Shared,
  Prime ...
  if cabs is empty then return "No cabs available, try later"
  for each cab in cabs do
    fare[cab] ← ESTIMATE-FARE(cab, pickup, destination)
    eta[cab] ← ESTIMATE-ARRIVAL-TIME(cab, pickup)
    utility[cab] ← w1*COMFORT(cab) − w2*fare[cab] − w3*eta[cab]
  end for
  options ← FILTER(cabs, preference)     // customer's cab type / budget
  chosen ← argmax over options of utility[cab]
  show options and chosen cab to the customer
  customer confirms or selects another cab type
  if BOOK(chosen) fails then
    remove chosen from options; select next best cab; repeat booking
  problem ← (initial = pickup, goal = destination, cost = time + fare)
  route ← A*-SEARCH(problem)             // or Uniform-Cost Search
  for each step in route do
    EXECUTE(step); UPDATE-STATE(GPS)
    if traffic changes then route ← A*-SEARCH(problem from current
    location)
  end for
  if CURRENT-LOCATION = destination then
    END-TRIP(); CALCULATE-FARE(); PAYMENT(); return "Trip completed"
```

7. Interpretation

The agent follows the cycle **formulate goal → formulate problem → search → execute**. The utility function lets the customer's comfort preference decide between cab types, while the search algorithm finds the least-cost route to the destination.

Question 5: Uniform Cost Search (UCS) – Least-Cost Delivery Path

1. Problem Understanding

Find the least-cost path from warehouse **S** to warehouse **G** in the given directed weighted graph, where edge weights represent fuel + time cost.

Edge	S→A	S→G	A→B	A→C	B→D	C→D	C→G	D→G
Cost	1	12	3	1	3	1	2	3

2. Method Selection

Uniform Cost Search expands the node with the lowest path cost g(n) from a priority queue. It is optimal and complete when all edge costs are positive, so it guarantees the minimum-cost route, unlike BFS (fewest edges) or DFS.

3. Solution Process (Step by Step)

g(n) = cost from S to node n. The frontier is a priority queue ordered by g(n).

Step	Node expanded (g)	Successors generated	Frontier after step (sorted by g)	Explored
1	S (0)	A = 0+1 = 1; G = 0+12 = 12	A(1), G(12)	S
2	A (1)	B = 1+3 = 4; C = 1+1 = 2	C(2), B(4), G(12)	S, A
3	C (2)	D = 2+1 = 3; G = 2+2 = 4 (better than 12, so G is updated)	D(3), B(4), G(4)	S, A, C
4	D (3)	G = 3+3 = 6 (worse than 4, so it is ignored)	B(4), G(4)	S, A, C, D
5	B (4)	D = 4+3 = 7 (worse than 3, so it is ignored)	G(4)	S, A, C, D, B
6	G (4)	Goal test passed, so the search stops	–	S, A, C, D, B, G

Note: B(4) and G(4) are tied; B was inserted earlier, so it is expanded first (FIFO tie-break). If G were taken first, the result would be the same.

4. Accuracy Check – All Possible Paths

Path	Cost calculation	Total cost
S → A → C → G	1 + 1 + 2	**4 (minimum)**
S → A → C → D → G	1 + 1 + 1 + 3	6
S → A → B → D → G	1 + 3 + 3 + 3	10
S → G	12	12

5. Final Answer

Optimal path: S → A → C → G with a total minimum cost of 4 units.

Order of node expansion: S, A, C, D, B, G.

6. Interpretation of Results

- The least-cost route costs **4 units** (fuel + time). The direct route S → G costs 12 units, so the optimal route saves **8 units (about 67%)**.
- The direct edge looks attractive because it has only one hop, but UCS considers cost and not the number of edges, so it correctly prefers the cheaper multi-hop route.
- UCS is optimal because it always expands the cheapest node first (all costs are positive), so when G is selected for expansion, no cheaper path to G can exist.
- Complexity: O(b^(1 + ⌊ C*/ε⌋)) in time and space, where C* is the optimal cost (4) and ε is the minimum edge cost (1).